

DIGITAL LOGIC DESIGN

By
Ms.Saba Iqbal



PART 1: LOGIC DESIGN

- Introduction to number systems and arithmetic operations in binary system.
- Combinational circuits:
 - Logic Gates (AND, OR, NOT, NOR, NAND and XOR), in this regards we will give the truth tables and symbols for each .
 - Laws of Boolean algebra, deriving logical expression and simplification.
 - Karnaugh maps and its use for simplification .
 - half and full adders and binary coded decimal adders

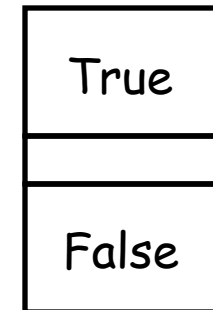


BOOLEAN OPERATIONS

- So far, we've talked about how arbitrary numbers can be represented using just the two binary values 1 and 0.
- Now we'll interpret voltages as the logical values “true” and “false” instead. We'll show:
 - How logical functions can be defined for expressing computations
 - How to build circuits that implement our functions in hardware

BOOLEAN VALUES

- Use electrical voltages to represent two discrete values 1 and 0, from which **binary numbers** can be formed.
- It's also possible to think of voltages as representing two *logical* values, **true** and **false**.
- For simplicity, we often still write digits instead:
 - **1** is true
 - **0** is false
- We will use this interpretation along with special operations to design functions and hardware for doing arbitrary computations.



FUNCTIONS

- Computers take inputs and produce outputs, just like functions in math!

An **expression** is
finite but not unique

$$f(x,y) = 2x +$$

y

$$= x + x + y$$

$$= 2(x + y/2)$$

$$= \dots$$

A **function table** is
unique but infinite

x	y	f(x,y)
0	0	0
...	...	...
2	2	6
...	...	...
23	41	87
...	...	...

- Logical functions can be expressed in two ways:
 - A finite, but non-unique Boolean expression.
 - A truth table, which will turn out to be unique and finite.

BASIC BOOLEAN OPERATIONS

- There are three basic operations for logical values.

Operation:

AND (product)
of two inputs

OR (sum) of
two inputs

NOT (complement)
on one input

Expression:

xy , or $x \cdot y$

$x + y$

x'

Truth table:

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

x	x'
0	1
1	0

BOOLEAN EXPRESSIONS

- We can use these basic operations to form more complex expressions:

$$f(x,y,z) = (x + y')z + x'$$

- Some terminology and notation:
 - f is the name of the function.
 - (x,y,z) are the **input variables**, each representing 1 or 0. Listing the inputs is optional, but sometimes helpful.
 - A **literal** is any occurrence of an input variable or its complement. The function above has four literals: x , y' , z , and x' .
- Precedences are important, but not too difficult.
 - NOT has the highest precedence, followed by AND, and then OR.
 - Fully parenthesized, the function above would be kind of messy:

$$f(x,y,z) = (((x + (y'))z) + x')$$

TRUTH TABLES

- A **truth table** shows all possible inputs and outputs of a function.
- Remember that each input variable represents either 1 or 0.
 - Because there are only a finite number of values (1 and 0), truth tables themselves are finite.
 - A function with n variables has 2^n possible combinations of inputs.
- Inputs are listed in binary order—in this example, from 000 to 111.

$$f(x,y,z) = (x + y')z + x'$$

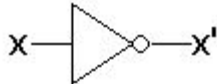
$$\begin{array}{llll} f(0,0,0) & = (0 + 1)0 + 1 & = & 1 \\ f(0,0,1) & = (0 + 1)1 + 1 & = & 1 \\ f(0,1,0) & = (0 + 0)0 + 1 & = & 1 \\ f(0,1,1) & = (0 + 0)1 + 1 & = & 1 \\ f(1,0,0) & = (1 + 1)0 + 0 & = & 0 \end{array}$$



x	y	z	f(x,y,z)
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

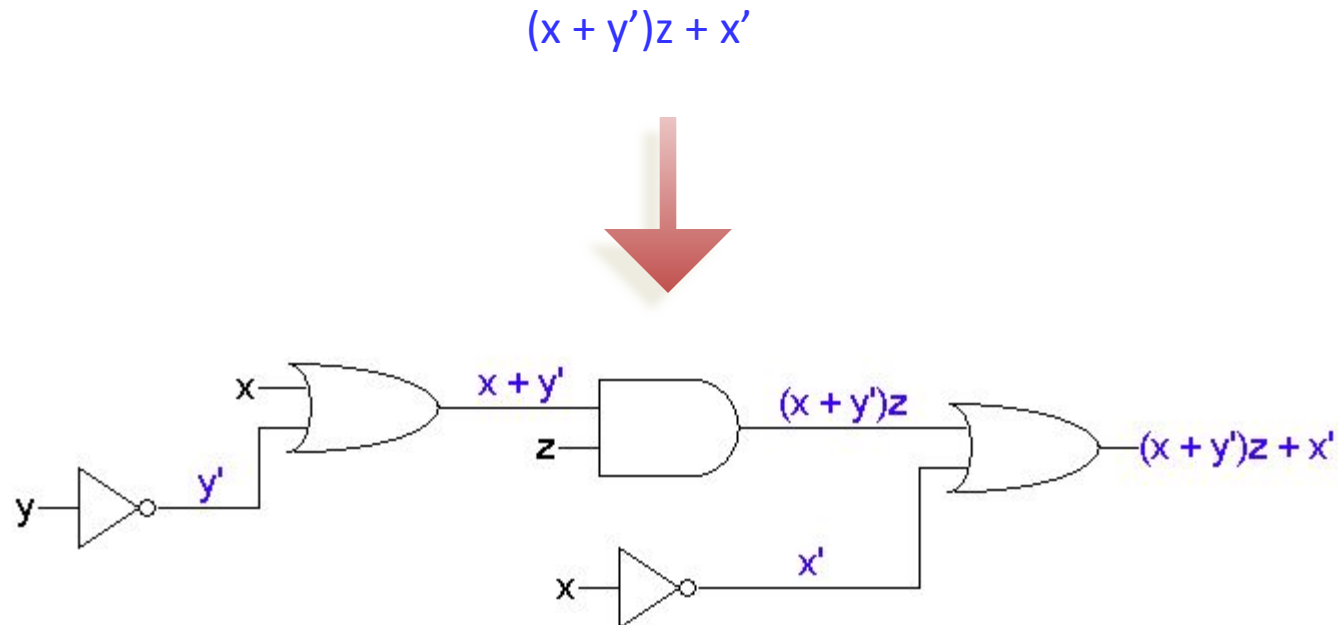
PRIMITIVE LOGIC GATES

- Each of our basic operations can be implemented in hardware using a **primitive logic gate**.
 - Symbols for each of the logic gates are shown below.
 - These gates output the product, sum or complement of their inputs.

Operation:	AND (product) of two inputs	OR (sum) of two inputs	NOT (complement) on one input
Expression:	xy , or $x \cdot y$	$x + y$	x'
Logic gate:			

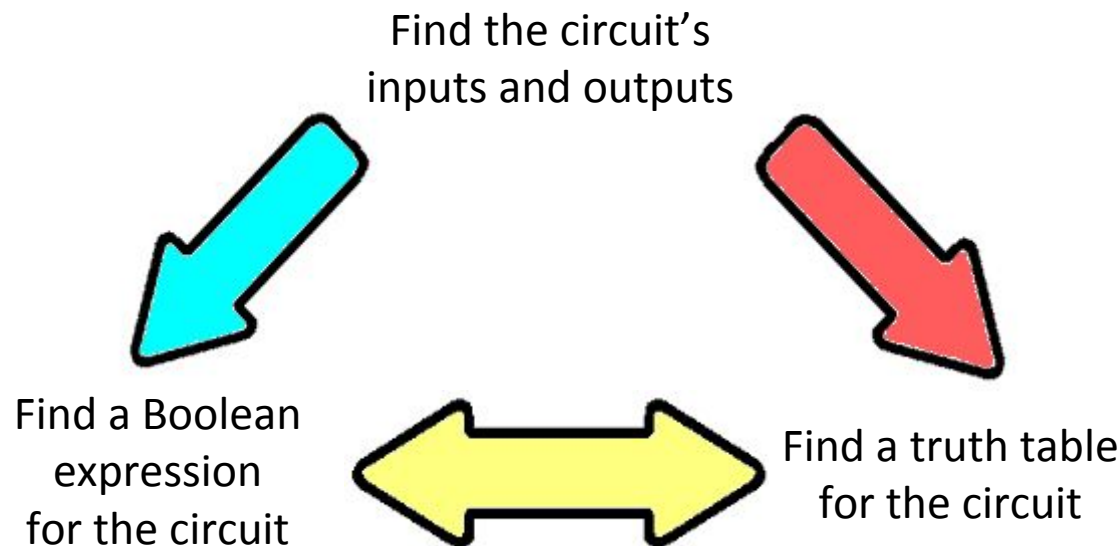
EXPRESSIONS AND CIRCUITS

- Any Boolean expression can be converted into a **circuit** by combining basic gates in a relatively straightforward way.
- The diagram below shows the inputs and outputs of each gate.
- The precedences are explicit in a circuit. Clearly, we have to make sure that the hardware does operations in the right order!



CIRCUIT ANALYSIS SUMMARY

- After finding the circuit inputs and outputs, you can come up with either an expression or a truth table to describe what the circuit does.
- You can easily convert between expressions and truth tables.



BOOLEAN OPERATIONS SUMMARY

- A variable whose value can be either 1 or 0 is called a Boolean variable.
 - AND, OR, and NOT are the basic Boolean operations.
 - We can express Boolean functions with either an expression or a truth table.
 - Every Boolean expression can be converted to a circuit.
-
- Next, we'll look at how Boolean algebra can help simplify expressions, which in turn will lead to simpler circuits.

BOOLEAN ALGEBRA

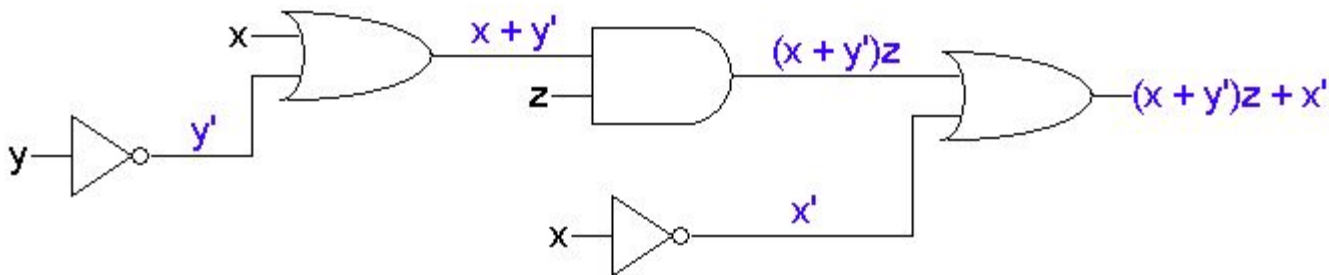
- Now we'll learn how to use Boolean algebra to simplify Booleans expressions.
- this expression and converted it to a circuit:

$$(x + y')z + x'$$



Can we make this circuit “better”?

- Cheaper: fewer gates
- Faster: fewer delays from inputs to outputs



EXPRESSION SIMPLIFICATION

- Normal mathematical expressions can be simplified using the laws of algebra
- For binary systems, we can use **Boolean algebra**, which is superficially similar to regular algebra
- There are many differences, due to
 - having only two values (0 and 1) to work with
 - having a complement operation
 - the OR operation is not the same as addition

FORMAL DEFINITION OF BOOLEAN ALGEBRA

- A Boolean algebra requires
 - A set of elements **B**, which needs *at least* two elements (0 and 1)
 - Two binary (two-argument) operations OR and AND
 - A unary (one-argument) operation NOT
 - The **axioms** below must always be true (textbook, p. 42)
 - The **magenta axioms** deal with the complement operation
 - **Blue axioms** (especially 15) are different from regular algebra

$$1. \ x + 0 = x$$

$$2. \ x \bullet 1 = x$$

$$3. \ x + 1 = 1$$

$$4. \ x \bullet 0 = 0$$

$$5. \ x + x = x$$

$$6. \ x \bullet x = x$$

$$7. \ x + x' = 1$$

$$8. \ x \bullet x' = 0$$

$$9. \ (x')' = x$$

$$10. \ x + y = y + x$$

$$11. \ xy = yx$$

Commutative

$$12. \ x + (y + z) = (x + y) + z$$

$$13. \ x(yz) = (xy)z$$

Associative

$$14. \ x(y + z) = xy + xz$$

$$15. \ x + yz = (x + y)(x + z)$$

Distributive

$$16. \ (x + y)' = x'y'$$

$$17. \ (xy)' = x' + y'$$

DeMorgan's

COMMENTS ON THE AXIOMS

- The associative laws show that there is no ambiguity about a term such as $x + y + z$ or xyz , so we can introduce multiple-input primitive gates:



- The left and right columns of axioms are **duals**
 - exchange all ANDs with ORs, and 0s with 1s
- The dual of *any* equation is always true

1. $x + 0 = x$	2. $x \bullet 1 = x$	
3. $x + 1 = 1$	4. $x \bullet 0 = 0$	
5. $x + x = x$	6. $x \bullet x = x$	
7. $x + x' = 1$	8. $x \bullet x' = 0$	
9. $(x')' = x$		
10. $x + y = y + x$	11. $xy = yx$	Commutative
12. $x + (y + z) = (x + y) + z$	13. $x(yz) = (xy)z$	Associative
14. $x(y + z) = xy + xz$	15. $x + yz = (x + y)(x + z)$	Distributive
16. $(x + y)' = x'y'$	17. $(xy)' = x' + y'$	DeMorgan's

ARE THESE AXIOMS FOR REAL?

- We can show that these axioms are **valid**, given the definitions of AND, OR and NOT

x	y	xy
0	0	0
0	1	0
1	0	0
1	1	1

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

x	x'
0	1
1	0

- The first 11 axioms are easy to see from these truth tables alone. For example, $x + x' = 1$ because of the middle two lines below (where $y = x'$)

x	y	x+y
0	0	0
0	1	1
1	0	1
1	1	1

PROVING THE REST OF THE AXIOMS

- We can make up truth tables to prove (both parts of) DeMorgan's law
- For $(x + y)' = x'y'$, we can make truth tables for $(x + y)'$ and for $x'y'$

x	y	$x + y$	$(x + y)'$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

x	y	x'	y'	$x'y'$
0	0	1	1	1
0	1	1	0	0
1	0	0	1	0
1	1	0	0	0

- In each table, the columns on the left (x and y) are the inputs. The columns on the right are outputs.
- In this case, we only care about the columns in blue. The other “outputs” are just to help us find the blue columns.
- Since both of the columns in blue are the same, this shows that $(x + y)'$ and $x'y'$ are equivalent

SIMPLIFICATION WITH AXIOMS

□ We can now start doing some simplifications

$$x'y' + xyz + x'y$$

$$= x'(y' + y) + xyz \quad [\text{Distributive; } x'y' + x'y = x'(y' + y)]$$

$$= x' \cdot 1 + xyz \quad [\text{Axiom 7; } y' + y = 1]$$

$$= x' + xyz \quad [\text{Axiom 2; } x' \cdot 1 = x']$$

$$= (x' + x)(x' + yz) \quad [\text{Distributive}]$$

$$= 1 \cdot (x' + yz) \quad [\text{Axiom 7; } x' + x = 1]$$

$$= x' + yz \quad [\text{Axiom 2}]$$

$$1. \quad x + 0 = x$$

$$2. \quad x \bullet 1 = x$$

$$3. \quad x + 1 = 1$$

$$4. \quad x \bullet 0 = 0$$

$$5. \quad x + x = x$$

$$6. \quad x \bullet x = x$$

$$7. \quad x + x' = 1$$

$$8. \quad x \bullet x' = 0$$

$$9. \quad (x')' = x$$

$$10. \quad x + y = y + x$$

$$11. \quad xy = yx$$

Commutative

$$12. \quad x + (y + z) = (x + y) + z$$

$$13. \quad x(yz) = (xy)z$$

Associative

$$14. \quad x(y + z) = xy + xz$$

$$15. \quad x + yz = (x + y)(x + z)$$

Distributive

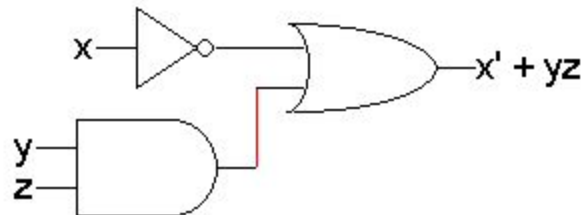
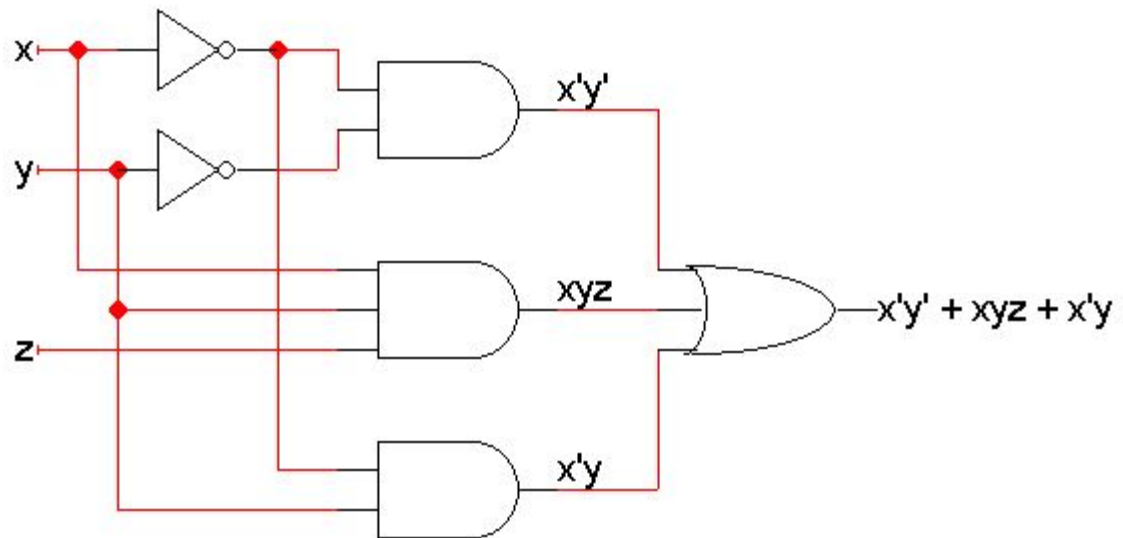
$$16. \quad (x + y)' = x'y'$$

$$17. \quad (xy)' = x' + y'$$

DeMorgan's

LET'S COMPARE THE RESULTING CIRCUITS

- Here are two different but *equivalent* circuits.
- In general the one with fewer gates is “better”:
 - It costs less to build
 - It requires less power
 - But we had to do some work to find the second form



SOME MORE LAWS

- Here are some more useful laws. Notice the duals again!

1. $x + xy = x$	4. $x(x + y) = x$
2. $xy + xy' = x$	5. $(x + y)(x + y') = x$
3. $x + x'y = x + y$	6. $x(x' + y) = xy$
$xy + x'z + yz = xy + x'z$	$(x + y)(x' + z)(y + z) = (x + y)(x' + z)$

- We can prove these laws by either

- Making truth tables:

x	y	x'	x'y	$x + x'y$
0	0			
0	1			
1	0			
1	1			

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1

- Using the axioms:

$$\begin{aligned}
 x + x'y &= (x + x')(x + y) \text{ [Distributive]} \\
 &= 1 \cdot (x + y) \quad [x + x' = 1] \\
 &= x + y \quad [\text{Axiom 3}]
 \end{aligned}$$

THE COMPLEMENT OF A FUNCTION

- The complement of a function always outputs 0 where the original function outputted 1, and 1 where the original produced 0.
- In a truth table, we can just exchange 0s and 1s in the output column(s)

$$f(x,y,z) = x(y'z' + yz)$$

x	y	z	$f(x,y,z)$
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	0



x	y	z	$f'(x,y,z)$
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	1

COMPLEMENTING A FUNCTION ALGEBRAICALLY

- You can use DeMorgan's law to keep "pushing" the complements inwards

$$f(x,y,z) = x(y'z' + yz)$$

$$f'(x,y,z) = (x(y'z' + yz))' \quad [\text{complement both sides}]$$

$$= x' + (y'z' + yz)' \quad [\text{because } (xy)' = x' + y']$$

$$= x' + (y'z')' (yz)' \quad [\text{because } (x + y)' = x' y']$$

$$= x' + (y + z)(y' + z') \quad [\text{because } (xy)' = x' + y', \text{ twice}]$$

- You can also take the dual of the function, and then complement each literal

- If $f(x,y,z) = x(y'z' + yz)$...
- ... the dual of f is $x + (y' + z')(y + z)$...
- ... then complementing each literal gives $x' + (y + z)(y' + z')$...
- ... so $f'(x,y,z) = x' + (y + z)(y' + z')$

SUMMARY SO FAR

- So far:
 - A bunch of Boolean algebra trickery for simplifying expressions and circuits
 - The algebra guarantees us that the simplified circuit is *equivalent* to the original one

- Next:
 - Introducing some standard forms and terminology
 - An alternative simplification method
 - We'll start using all this stuff to build and analyze bigger, more useful, circuits

BASIC CONCEPTS

□ Simple gates

- AND
- OR
- NOT

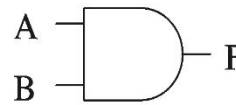
□ Functionality can be expressed by a truth table

- A truth table lists output for each possible input combination

□ Precedence

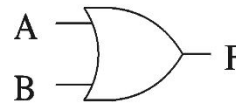
- NOT > AND > OR
- $F = A \underline{B} + \underline{A} B$
 $= (A (\underline{B})) + ((\underline{A}) B)$

I



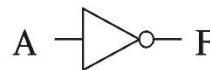
AND gate

A	B	F
0	0	0
0	1	0
1	0	0
1	1	1



OR gate

A	B	F
0	0	0
0	1	1
1	0	1
1	1	1



NOT gate

A	F
0	1
1	0

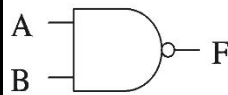
Logic symbol

Truth table

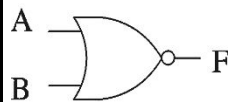


BASIC CONCEPTS (CONT.)

- Additional useful gates
 - NAND
 - NOR
 - XOR
- $\text{NAND} = \text{AND} + \text{NOT}$
- $\text{NOR} = \text{OR} + \text{NOT}$
- XOR implements exclusive-OR function
- NAND and NOR gates require only 2 transistors
 - AND and OR need 3 transistors!



NAND gate



NOR gate



XOR gate

Logic symbol

A	B	F
0	0	1
0	1	1
1	0	1
1	1	0

A	B	F
0	0	1
0	1	0
1	0	0
1	1	0

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

Truth table

BASIC CONCEPTS (CONT.)

□ Number of functions

- With N logical variables, we can define 2^{2N} functions
- Some of them are useful
 - AND, NAND, NOR, XOR, ...
- Some are not useful:
 - Output is always 1
 - Output is always 0
- “Number of functions” definition is useful in proving completeness property



BASIC CONCEPTS (CONT.)

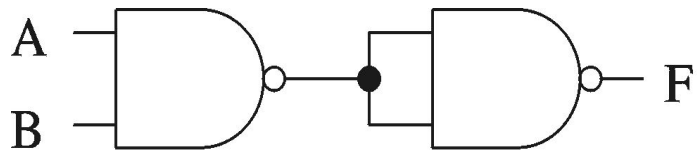
□ Complete sets

- A set of gates is complete
 - If we can implement any logical function using only the type of gates in the set
 - You can use as many gates as you want
- Some example complete sets
 - {AND, OR, NOT} ← Not a minimal complete set
 - {AND, NOT}
 - {OR, NOT}
 - {NAND}
 - {NOR}
- Minimal complete set
 - A complete set with no redundant elements.

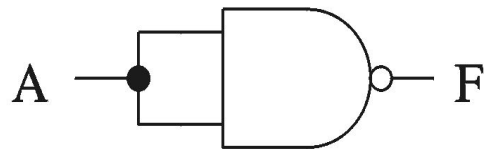


BASIC CONCEPTS (CONT.)

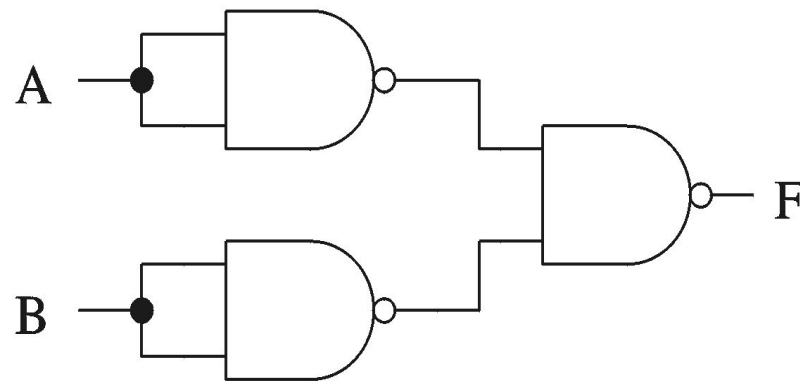
- Proving NAND gate is universal
- NAND gate is called *universal gate*



AND gate



NOT gate

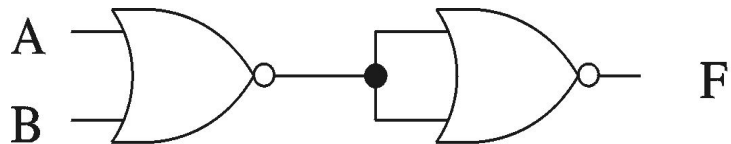


OR gate

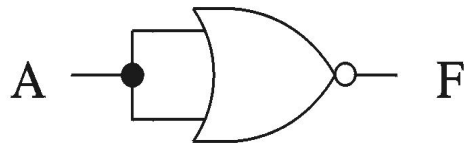


BASIC CONCEPTS (CONT.)

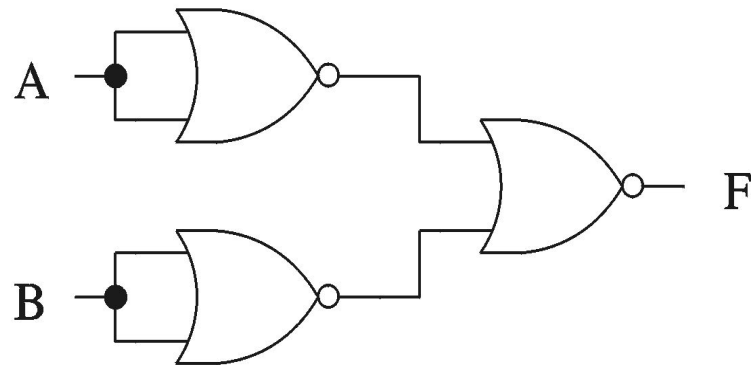
- Proving NOR gate is universal
- NOR gate is called *universal gate*



OR gate



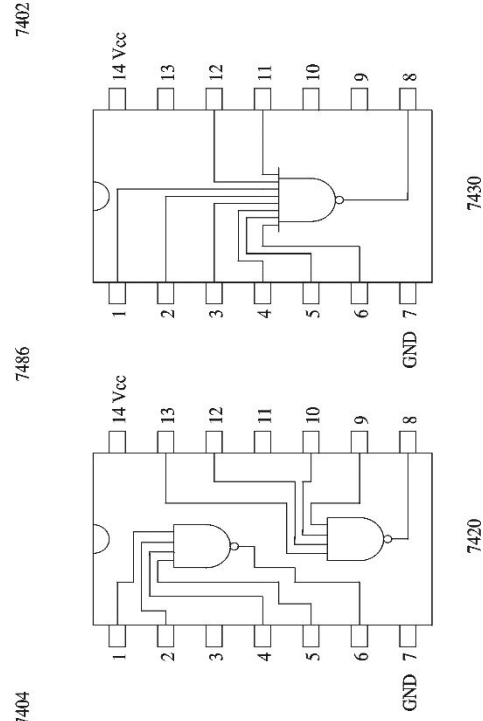
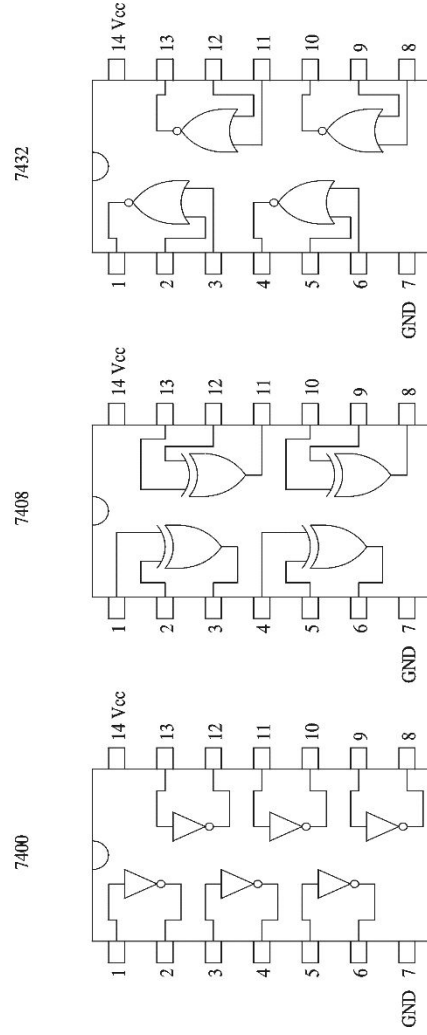
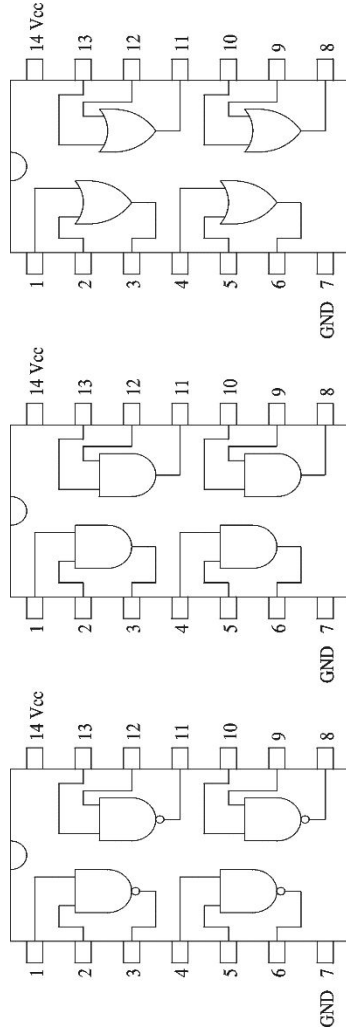
NOT gate



AND gate



LOGIC CHIPS



LOGIC FUNCTIONS

- Logical functions can be expressed in several ways:
 - Truth table
 - Logical expressions
 - Graphical form
- Example:
 - Majority function
 - Output is 1 whenever majority of inputs is 1
 - We use 3-input majority function



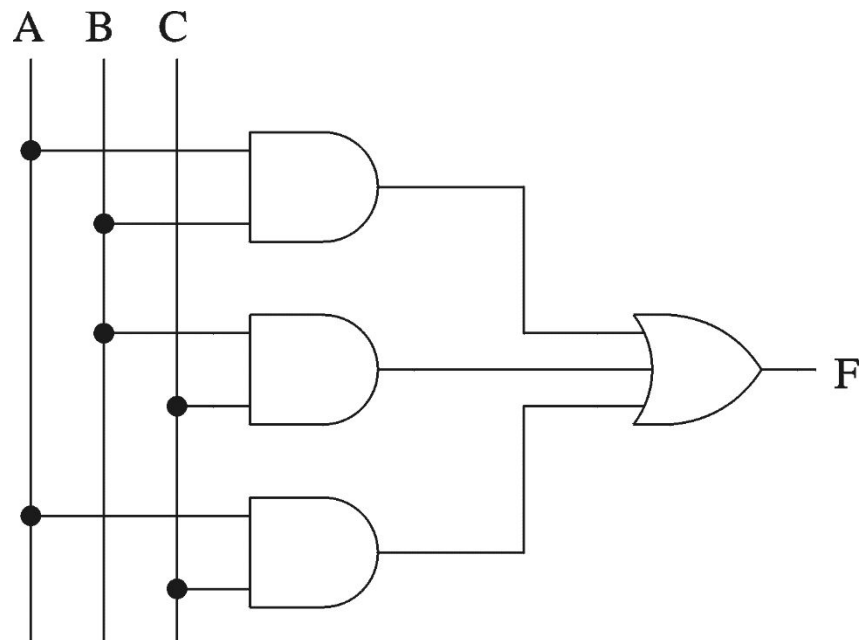
LOGIC FUNCTIONS (CONT.)

3-input majority function

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

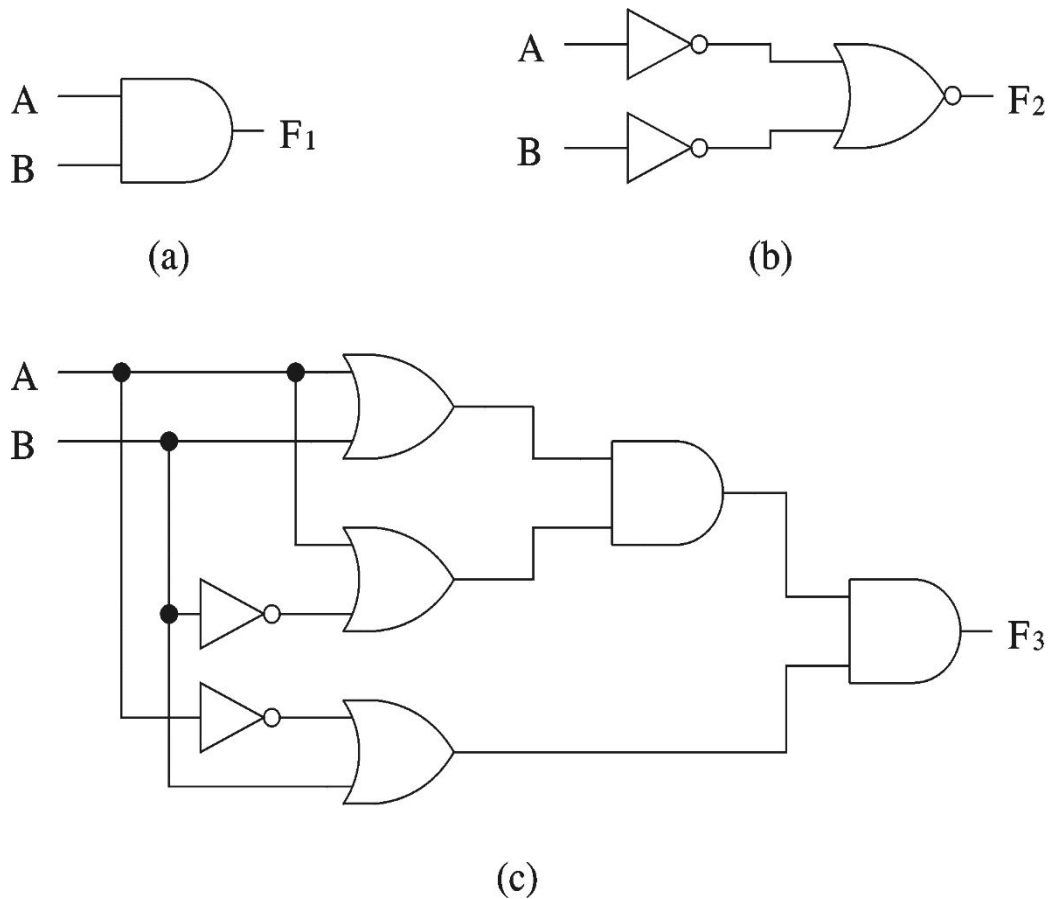
□ Logical expression form

$$F = A B + B C + A C$$



LOGICAL EQUIVALENCE

- All three circuits implement $F = A B$ function



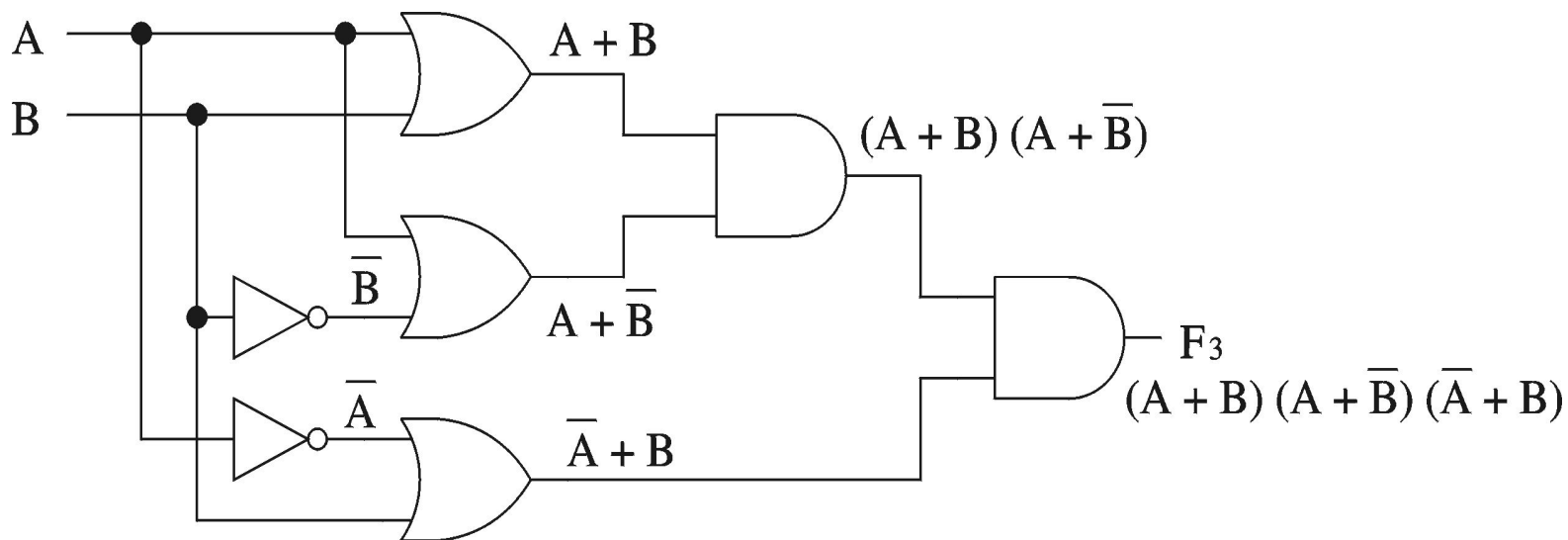
LOGICAL EQUIVALENCE (CONT.)

- Proving logical equivalence of two circuits
 - Derive the logical expression for the output of each circuit
 - Show that these two expressions are equivalent
 - Two ways:
 - You can use the truth table method
 - For every combination of inputs, if both expressions yield the same output, they are equivalent
 - Good for logical expressions with small number of variables
 - You can also use algebraic manipulation
 - Need Boolean identities



LOGICAL EQUIVALENCE (CONT.)

- Derivation of logical expression from a circuit
 - Trace from the input to output
 - Write down intermediate logical expressions along the path



LOGICAL EQUIVALENCE (CONT.)

- Proving logical equivalence: Truth table method

A	B	$F1 = A \cdot B$	$F3 = (A + B) \cdot (A + B)$	—	—
0	0	0	0		
0	1	0	0		
1	0	0	0		
1	1	1	1		



DERIVING LOGICAL EXPRESSIONS

- Derivation of logical expressions from truth tables
 - sum-of-products (SOP) form
 - product-of-sums (POS) form
- SOP form
 - Write an AND term for each input combination that produces a 1 output
 - Write the variable if its value is 1; complement otherwise
 - OR the AND terms to get the final expression
- POS form
 - Dual of the SOP form



DERIVING LOGICAL EXPRESSIONS (CONT.)

- 3-input majority function

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

- SOP logical expression

- Four product terms

- Because there are 4 rows with a 1 output

$$F = A B C + A B \overline{C} + \overline{A} B C + A \overline{B} C$$



DERIVING LOGICAL EXPRESSIONS (CONT.)

□ 3-input majority function

□ POS logical expression

□ Four sum terms

- Because there are 4 rows with a 0 output

$$F = (A + B + C) (A + \overline{B} + C) (A + \overline{B} + \overline{C}) (A + B + \overline{C})$$

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

LOGICAL EXPRESSION SIMPLIFICATION

- Two basic methods
 - Algebraic manipulation
 - Use Boolean laws to simplify the expression
 - Difficult to use
 - Don't know if you have the simplified form
 - Karnaugh map (K-map) method
 - Graphical method
 - Easy to use
 - Can be used to simplify logical expressions with a few variables



ALGEBRAIC MANIPULATION

- Majority function example

$$\overline{A}BC + A\overline{B}C + ABC + \overline{A}B\overline{C} =$$

$$ABC + A\overline{B}C + A\overline{B}C + A\overline{B}C + A\overline{B}C + A\overline{B}C + A\overline{B}C$$

Added extra

- We can now simplify this expression as

$$BC + AC + AB$$

- A difficult method to use for complex expressions



KARNAUGH MAP METHOD

Note the order

A \ B	0	1
0		
1		

(a) Two-variable K-map

A \ BC	00	01	11	10
0				
1				

(b) Three-variable K-map

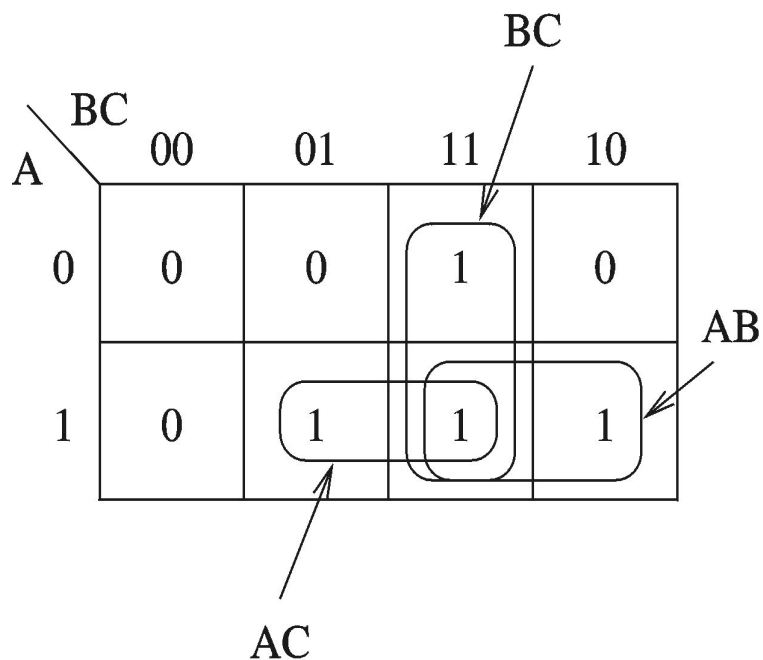
AB \ CD	00	01	11	10
00				
01				
11				
10				

(c) Four-variable K-map

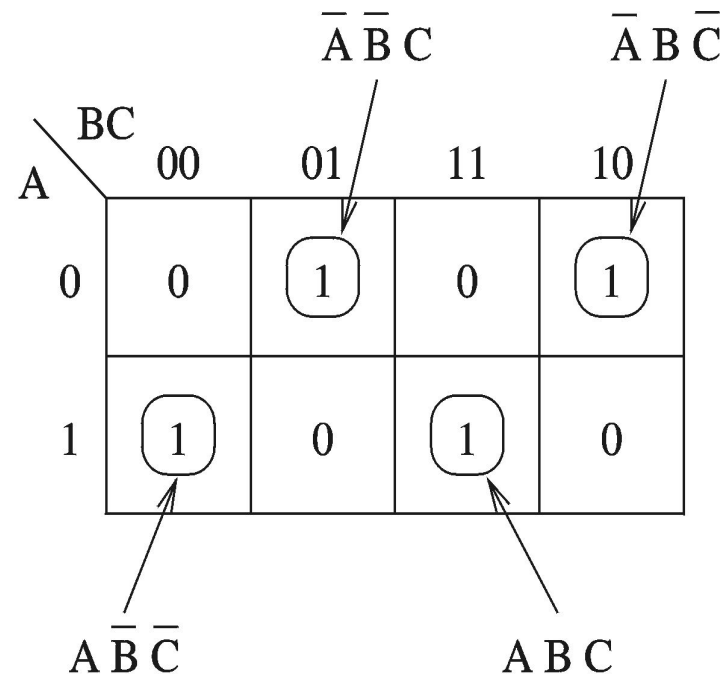


KARNAUGH MAP METHOD (CONT.)

Simplification examples



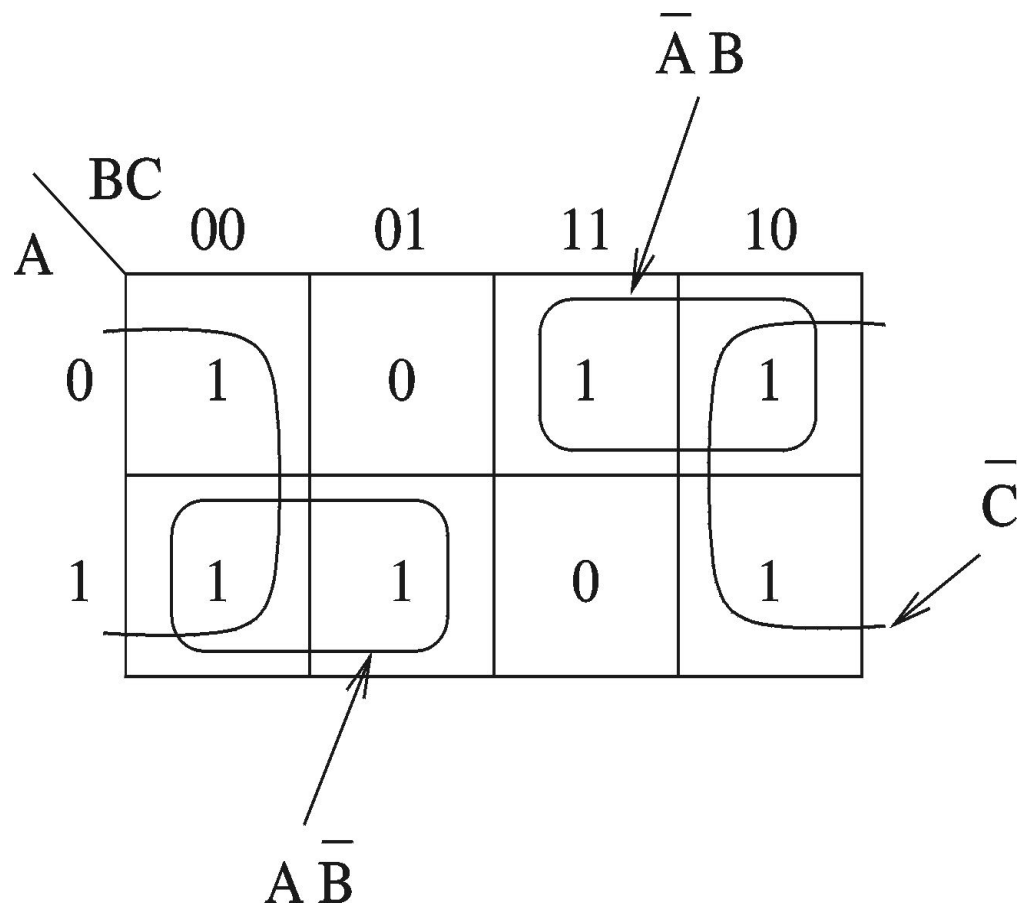
(a) Majority function



(b) Even-parity function

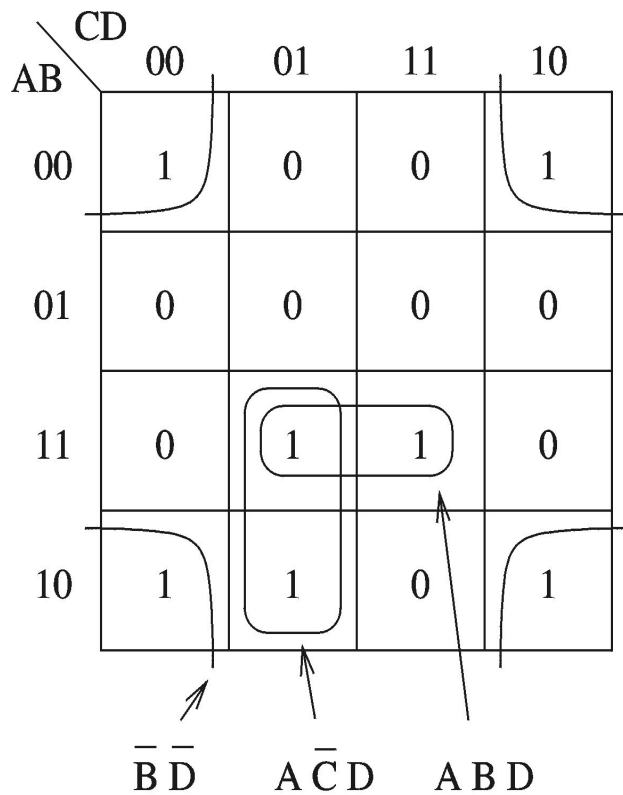
KARNAUGH MAP METHOD (CONT.)

First and last columns/rows are adjacent

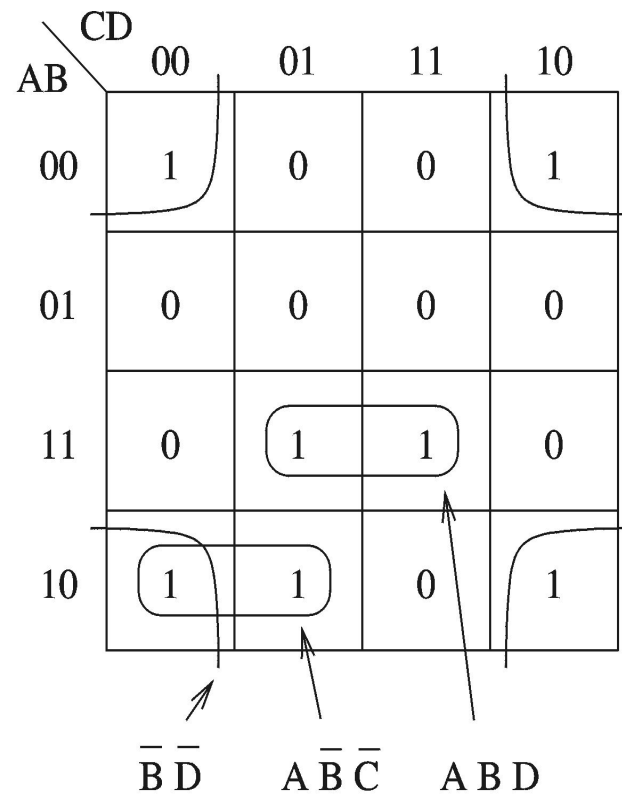


KARNAUGH MAP METHOD (CONT.)

Minimal expression depends on groupings



(a)



(b)



KARNAUGH MAP METHOD (CONT.)

No redundant groupings

AB \ CD	CD			
	00	01	11	10
00	0	0	1	0
01	1	1	1	0
11	0	1	1	1
10	0	1	0	0

(a) Nonminimal simplification

AB \ CD	CD			
	00	01	11	10
00	0	0	1	0
01	1	1	1	0
11	0	1	1	1
10	0	1	0	0

(b) Minimal simplification

